

A Third-Order Sigma-Delta Digital-to-Analog Converter

A multi-tool design and verification flow for telephony-band voice

Introduction

This document describes the design and verification of a third-order sigma-delta digital-to-analog converter (DAC) targeting telephony-band voice signals (300 Hz – 3.4 kHz). The converter accepts a 13-bit signed audio input at 8 kHz and produces a 1-bit pulse-density-modulated (PDM) output at 5 MHz, intended to be filtered by an external analog low-pass network to recover the analog audio waveform.

More than a circuit, the project is a controlled exercise in mixed-signal design flow. Each stage of the converter's development was deliberately carried out in the tool best suited to it: architectural synthesis in MATLAB with the Schreier Delta-Sigma Toolbox; bit-true conversion and mixed-signal modelling in Simulink with Fixed-Point Designer; synthesisable hardware description in Vitis HLS C++; and analog reconstruction simulation in LTspice. The handoffs between these tools — the floating-point reference function, the fixed-point variant, the HLS C++ source, and the PWL bitstream file — are themselves first-class design artefacts, archived alongside the source so that any stage of the flow can be reproduced in isolation.

The exercise was built around a concrete numerical target: roughly 13 bits of effective resolution (≈ 80 dB SINAD) across the voice band. This number drives every design decision in the document — the choice of modulator order, the oversampling ratio, the fixed-point widths, and the reconstruction filter topology — and gives the work a defensible engineering objective rather than a vague aspiration.

The document is organised to follow the design flow itself. It opens with the theoretical background and the choice of modulator architecture, then walks through the MATLAB design script, the Simulink mixed-signal model, the Fixed-Point Tool conversion, the Vitis HLS implementation, the LTspice analog verification with the PWL handoff, and the two reconstruction filter options. A concluding section summarises what was demonstrated and what would be needed to formally close the gap to the design target.

Scope and limitations

The 13-bit ENOB figure used as the design target throughout this document serves to motivate concrete decisions at each stage of the flow: modulator order, oversampling ratio, fixed-point widths, reconstruction filter topology. The simulations performed within this exercise do not constitute a formal end-to-end demonstration that this performance level is achieved in practice. The measured SINAD in the LTspice signal chain is bounded by simulation-environment factors — finite FFT length, solver tolerances, op-amp model fidelity, and audio-rate image artefacts — that mask the modulator's intrinsic capability. Closing the gap to the design target would require additional verification steps outlined in the conclusion, including direct MATLAB analysis of the modulator bitstream and a digital interpolation filter to suppress the audio-rate images. The primary deliverable of the work is therefore the demonstrated multi-tool flow itself, of which the document's remaining sections are the technical record.

Background

Operating principles

A sigma-delta DAC reconstructs a high-resolution analog signal from low-resolution digital data using three complementary techniques.

Oversampling. The converter operates at a sample rate far above the Nyquist frequency of the signal of interest. This spreads quantisation noise across a wide frequency band, reducing its spectral density inside the signal band.

Noise shaping. A feedback loop, the modulator, redistributes the remaining quantisation noise so that it is attenuated within the signal band and pushed up to higher frequencies where it can be filtered away.

Reconstruction filtering. A simple analog low-pass filter at the output removes the high-frequency shaped noise, leaving a clean, high-resolution analog signal.

Why sigma-delta

Sigma-delta converters routinely deliver 16 to 24 bits of effective resolution, well beyond what resistor-ladder (R-2R) architectures can achieve, since the latter are bounded by the matching tolerance of physical components. Most of the conversion is performed in the digital domain, so the design fits naturally into standard CMOS processes without requiring trimmed analog elements; the analog portion reduces to a quantiser, a few integrators, and a reconstruction filter. These properties make the architecture ubiquitous in audio, in precision instrumentation such as industrial weighing systems and temperature sensors, and in communications equipment.

Theoretical references

The design methodology adopted in this work draws on a small set of well-established references that together cover the theory, simulation, and practical implementation of delta-sigma converters.

Schreier and Temes, *Understanding Delta-Sigma Data Converters* (Wiley, 2nd ed.). This textbook is the standard reference in both academic and industrial practice. It bridges the mathematical foundations of noise shaping with the physical realities of circuit implementation and addresses modern topics including continuous-time modulators and multi-bit mismatch shaping.

Schreier's Delta-Sigma Toolbox for MATLAB. The toolbox provides validated scripts for noise-transfer-function synthesis, dynamic-range scaling, and high-level performance prediction, allowing the state-space techniques developed in the textbook to be applied directly without re-implementation. The toolbox is available through MATLAB Central File Exchange, listing 19.

Using these resources together yields several practical benefits for the design flow: pre-verified synthesis routines for high-order noise transfer functions, established stability criteria applied at the architectural stage, dynamic-range scaling functions that size integrator outputs to prevent saturation, quantitative performance prediction before any implementation work, and a behavioural reference against which fixed-point and transistor-level implementations can be verified. This methodology provides a defensible foundation for the design and a unified framework within which to balance power, area, and resolution.

Modulator architecture

Topology selection

The sigma-delta modulator is implemented as a third-order Cascade-of-Integrators with Distributed Feedback (CIFB) structure, synthesised using the Schreier Delta-Sigma Toolbox.

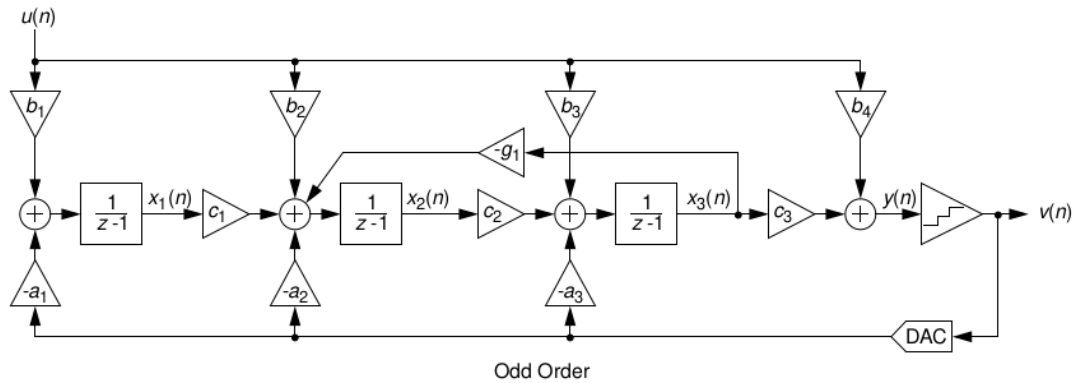


Figure 1. Third-order CIFB topology with local resonator feedback (g_1). The diagram shows the canonical odd-order CIFB structure used as the design template.

The CIFB topology was chosen for several reasons. First, distributing feedback across all integrators distributes signal swing more evenly than a feed-forward (CIFF) structure, which simplifies dynamic-range scaling and reduces the likelihood of any single integrator saturating under worst-case input. Second, the topology supports a local resonator feedback path — coefficient g_1 in the diagram — that relocates the noise transfer function (NTF) zeros from DC to non-zero frequencies within the signal band, providing flatter and lower in-band noise compared with a simple cascade of differentiators. Third, the structure maps cleanly onto an FPGA implementation: each integrator is a single accumulator with two feedback subtractions, and the resonator is one additional multiply-and-subtract per sample.

A third-order modulator with an oversampling ratio of 625 (5 MHz PDM rate, 8 kHz audio sample rate) and a 1-bit quantiser was chosen as the smallest configuration that comfortably exceeds the target SINAD. The floating-point reference simulation predicts approximately 104 dB SNR at the design input level, leaving substantial headroom above the 13-bit (≈ 80 dB) target. The Lee criterion is satisfied with a peak NTF gain of 1.5, ensuring stable operation, and the maximum stable input amplitude ($u_{\max} \approx 0.75$) provides margin above the chosen 0.5 normalised input scaling.

Design script automation

The MATLAB script `DESIGN_THIRD_ORDER.m` automates the synthesis, scaling, and golden-reference generation for the modulator.

```

1  %% 3rd-Order Sigma-Delta Design and Golden-Reference Generation
2  %% CIFB topology with one resonator local feedback (g1) for in-band NTF zeros
3  %% Produces the floating-point reference for MATLAB Fixed-Point Designer.
4  %% Requires: Delta-Sigma Toolbox
5  %% MATLAB R2021b compatible
6
7  clear; clc;
8
9  %% -----
10 % PART 1: DESIGN
11 % -----
12 order = 3;
13 osr = 625; % 5MHz / 8kHz
14 nlev = 2; % 1-bit quantizer
15 form = 'CIFB';
16 input_bits = 13; % signed 13-bit input
17 input_scale = 2^(input_bits-1); % = 4096
18 A_input = 0.5; % scaled input amplitude
19
20 fprintf('Input: %d-bit signed, range [%d, %d]\n',...
21         input_bits, -input_scale, input_scale-1);
22
23 %% 1) Synthesize NTF
24 %% Hinf = 1.5 is the Lee criterion limit. For 3rd-order modulators a slightly
25 %% relaxed Hinf (e.g. 2.0-3.0) is sometimes used to gain noise-shaping
26 %% aggressiveness; start at 1.5 and revisit only if SNR is insufficient.
27 H_inf = 1.5;
28 ntf = synthesizeNTF(order, osr, 1, H_inf);
29
30 %% 2) Realize coefficients (CIFB topology)
31 [a, g, b, c] = realizeNTF(ntf, form);
32
33 %% 3) Build ABCD matrix and scale dynamic range
34 ABCD = stuffABCD(a, g, b, c, form);
35 [ABCDs, umax] = scaleABCD(ABCD, nlev, [], A_input);
36

```

Figure 2. Header of *DESIGN_THIRD_ORDER.m* showing the design parameters: third-order CIFB, OSR = 625, 13-bit signed input, normalised input amplitude of 0.5.

The script implements the standard top-down sigma-delta design flow from Schreier and Temes, broken into five phases.

Phase 1 — NTF synthesis. Given the order, OSR, and a stability target H_∞ , `synthesizeNTF` finds the optimal noise transfer function: the one that minimises in-band noise power subject to staying stable. The optimisation places NTF zeros at frequencies that minimise the in-band integral of $|NTF(f)|^2$ over the signal band $[0, fs/(2 \cdot OSR)]$, while constraining the maximum out-of-band gain to H_∞ .

Phase 2 — Topology realisation. `realizeNTF(ntf, 'CIFB')` translates the abstract NTF into the loop-filter coefficients (a, g, b, c) for a specific physical structure. Several legal realisations of any given NTF exist — CIFB, CIFF, CRFB, CRFF — each with different signal/noise distribution, sensitivity to coefficient quantisation, and area/power trade-offs. CIFB was selected because it distributes signal swing more evenly across the integrators than CIFF.

Phase 3 — Dynamic-range scaling. `scaleABCD` simulates the modulator with a sine wave at amplitude `A_input` and rescales each integrator so its peak output stays bounded. Without this step, internal signals can swing 10–100 times larger than the input, which would force enormous accumulator widths in hardware. The scaling preserves the overall NTF (it is a state-space coordinate change) but redistributes signal swing.

Phase 4 — Verification. `simulateDSM` runs the modulator with a windowed sine input, computes the output FFT, and `calculateSNR` integrates noise power within the signal band. This gives the floating-point SNR ceiling, the best the architecture can do regardless of fixed-point implementation. `predictSNR` independently computes the theoretical maximum SNR from the NTF alone as a cross-check.

Phase 5 — Golden-reference generation. The script writes `SDM_fcn.m`, a bit-true floating-point implementation of the modulator. This file is the single point from which all downstream tools (Fixed-Point Designer, HLS) derive the modulator equations, ensuring complete consistency from architectural design through to hardware implementation.

Stability verification

The script verifies three separate stability conditions, each addressing a different failure mode.

1. NTF pole stability — a necessary condition

The NTF, considered as a discrete-time transfer function, must have all its poles strictly inside the unit circle in the z -plane. If any pole has magnitude greater than or equal to 1, the linearised loop is unstable and quantisation noise grows without bound. This is a necessary but not sufficient condition for sigma-delta stability: a modulator with stable NTF poles can still go unstable in the nonlinear sense because of the 1-bit quantiser, which is what the next two checks address.

2. Lee criterion — a heuristic sufficient condition

The Lee criterion (W. Lee, MIT, 1987) is the most widely used heuristic for sigma-delta stability: a 1-bit sigma-delta modulator is empirically stable if $\max|NTF(j\omega)| \leq 1.5$ (sometimes stated as ≤ 2 depending on the source). The reasoning rests on a quasi-linear model of the 1-bit quantiser as a variable-gain element whose effective gain depends on the input. Linearised stability requires the loop to remain stable across the range of gains the quantiser actually exhibits; Lee observed empirically that if the NTF's peak magnitude stays below 1.5, the modulator usually remains stable for typical inputs.

The criterion is a rule of thumb rather than a theorem. Some modulators with peak NTF gain of 2 are stable; some with 1.4 are not. But 1.5 is a defensible operating point on which the literature has converged. The current design hits the boundary exactly because `synthesizeNTF` was called with $H^\infty = 1.5$, which constrained the peak to that limit.

3. Maximum stable input — the practical boundary

`scaleABCD` simulates the modulator with progressively larger sine inputs and finds the largest amplitude $u_{\max}$ at which it remains stable, defined here as integrator excursions remaining bounded over a long simulation. For this design, $u_{\max} \approx 0.747$.

The Lee criterion gives a stable NTF; $u_{\max}$ tells you the input amplitude that NTF can actually handle. These are separate things: a modulator with $H^\infty = 1.5$ typically has $u_{\max}$ around 0.7–0.8 of full-scale; with $H^\infty = 2.0$, more aggressive noise shaping but $u_{\max}$ typically drops to 0.5–0.6; with $H^\infty = 3.0$, $u_{\max}$ may fall as low as 0.3–0.4. This is the fundamental trade-off in 1-bit sigma-delta design: better in-band noise performance costs you input range. The relationship was confirmed experimentally during LTspice testing, where feeding an input close to $u_{\max} \times 4096$ caused the modulator to enter an unstable mode, exactly as the theory predicts.

Simulink mixed-signal verification

Model overview

A Simulink model was built to verify the modulator end-to-end in a mixed digital/analog environment, combining the discrete-time sigma-delta function with a continuous-time analog reconstruction stage.

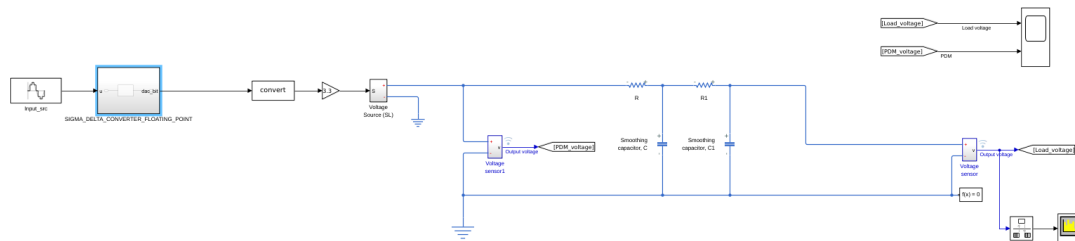


Figure 3. Simulink mixed-signal verification model. The digital path on the left passes through the SIGMA_DELTA_CONVERTER_FLOATING_POINT subsystem; the analog path on the right uses a Simscape-Electrical RC reconstruction filter with voltage sensors tapping the PDM and load nodes.

The digital path begins with an Input_src signal generator that produces the test stimulus, typically a 13-bit signed sine wave. This feeds the SIGMA_DELTA_CONVERTER_FLOATING_POINT block, an embedded MATLAB Function that wraps the auto-generated SDM_fcn.m produced by DESIGN_THIRD_ORDER.m. The block runs the same floating-point modulator equations used as the golden reference for Fixed-Point Designer, ensuring complete consistency between the architectural design, the bit-true reference, and the Simulink simulation.

The block's dac_bit output (logical 0 or 1) passes through a convert block to a numeric type and is scaled by 3.3 V to represent the physical voltage swing at the FPGA's output pin. A Simscape-Electrical Voltage Source (SL) injects this signal into the analog domain. The reconstruction filter is implemented as a passive second-order RC ladder using two resistors (R, R1) and two smoothing capacitors (C, C1), modelling the analog low-pass that removes the high-frequency shaped quantisation noise. Two voltage-sensor blocks tap the PDM signal before the filter and the load voltage after it, routing both to a scope via PDM_voltage and Load_voltage From/Goto tags so that input and reconstructed waveforms can be compared directly in the time domain. The f(x) = 0 solver-configuration block and the simulation-output blocks complete the Simscape network.

Fixed-point conversion

```

17 function dac_bit = SDM_fcn(u) %#codegen
18
19 dac_bit = false;
20
21 persistent int1 int2 int3
22 if isempty(int1)
23     int1 = 0;
24     int2 = 0;
25     int3 = 0;
26 end
27
28 % Floating-point coefficients (scaled CIFB, 3rd order)|
29 B1 = 0.172222623529955;
30 A1 = 0.172222623529955;
31 A2 = 0.123562088507108;
32 A3 = 0.122457379474687;
33 C1 = 0.10952288682635;
34 C2 = 0.357050181625205;
35 G1 = 4.2457997600222e-05;
36
37 % Saturate and normalise the 13-bit signed input to [-1, +1)
38 u_sat = max(min(double(u), 4095), -4096);
39 u_norm = u_sat / 4096;
40
41 % Quantizer decision based on the last integrator (int3)
42 if int3 >= 0
43     dac_bit = true;
44     v = 1;
45 else
46     dac_bit = false;
47     v = -1;
48 end
49
50 % Integrator 1: int1 += B1*u - A1*v
51 int1 = int1 + B1*u_norm - A1*v;
52
53 % Integrator 2: int2 += C1*int1 - A2*v
54 int2 = int2 + C1*int1 - A2*v;
55
56 % Integrator 3: int3 += C2*int2 - G1*int3 - A3*v
57 % G1 is the resonator local feedback that places NTF zeros
58 % at non-DC frequencies inside the signal band.
59 int3 = int3 + C2*int2 - G1*int3 - A3*v;
60
61 end

```

Figure 4. Floating-point reference *SDM_fcn.m* as it appears inside the *SIGMA_DELTA_CONVERTER_FLOATING_POINT* subsystem. The function uses double-precision arithmetic so that Fixed-Point Designer can instrument the model and observe each variable's dynamic range.

The floating-point modulator function *SDM_fcn.m* was converted to a fixed-point implementation using MATLAB's Fixed-Point Tool, accessible directly from within Simulink. The conversion was performed on the *SIGMA_DELTA_CONVERTER_FLOATING_POINT* subsystem, which wraps the modulator function as a Simulink MATLAB Function block. Running the "Convert to Fixed Point" workflow instruments the model, simulates it against the chosen test stimulus, observes the dynamic range of every internal signal, and proposes appropriate fixed-point types (sign, word length, fraction length) for each variable: coefficients, integrator states, intermediate products, and the input/output ports.

After accepting the proposed types, the tool automatically generates a parallel implementation of the function as a variant subsystem: the original *SDM_fcn* block is preserved as the floating-point reference (MATLAB Function variant), and a new fixed-point version, *SDM_fcn_fixpt*, is created alongside it (MATLAB Function_FixPt variant). This dual-variant structure allows the

model to switch between the two implementations through the Simulink variant-control mechanism, so that the same model can be simulated either as a bit-true reference or as a fixed-point candidate without changing the surrounding analog and stimulus blocks.

The fixed-point variant uses the types proposed by the tool — `fi(1, 16, ·)` formats for signed values and `fi(0, 16, ·)` for the unsigned coefficients — with `fimath` settings of Floor rounding and Wrap overflow to match the eventual hardware behaviour. The floating-point block remains untouched and editable: any future redesign re-runs the script, regenerates `SDM_fcn.m`, and triggers a fresh conversion, while the fixed-point block is regenerated automatically and any manual edits to it are overwritten on the next conversion. This separation keeps the design intent (the equations) and the implementation choices (the bit widths) cleanly decoupled, providing a verifiable bridge between the MATLAB design phase and the Vitis HLS implementation phase that follows.

Time-domain verification

With the fixed-point variant active, the model was simulated and the PDM and reconstructed waveforms were observed on the scope.

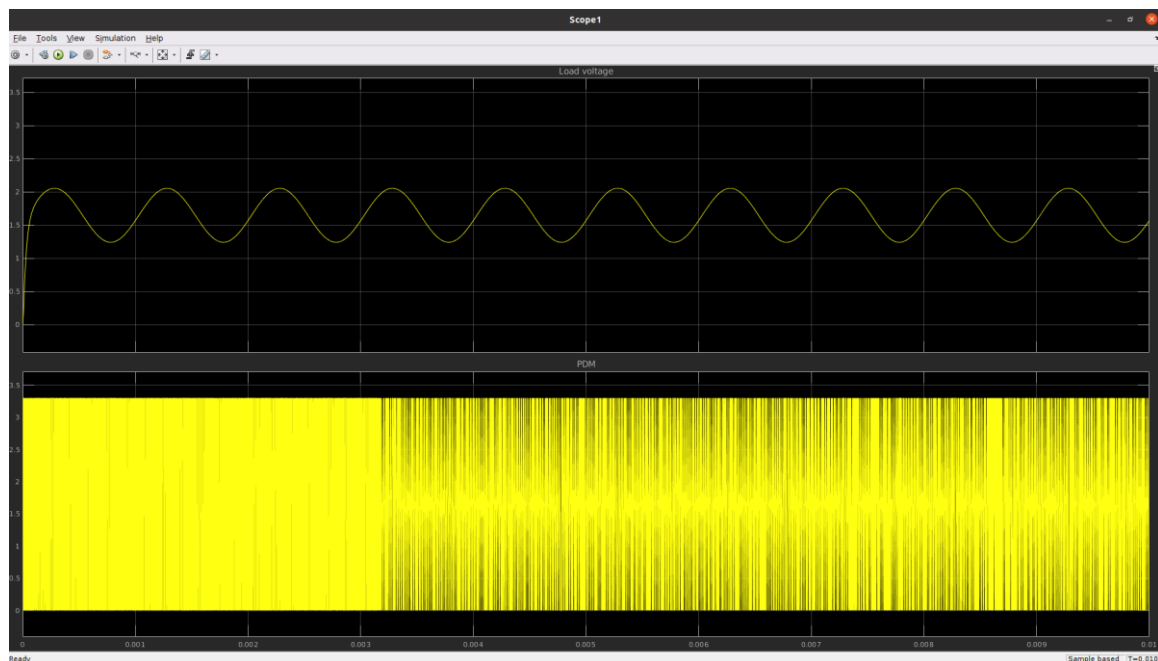


Figure 5. Simulated PDM bitstream (lower trace) and reconstructed analog audio at the load (upper trace). The high-density PDM signal is filtered by the Simscape RC network into a clean sinusoid centred at the supply mid-point.

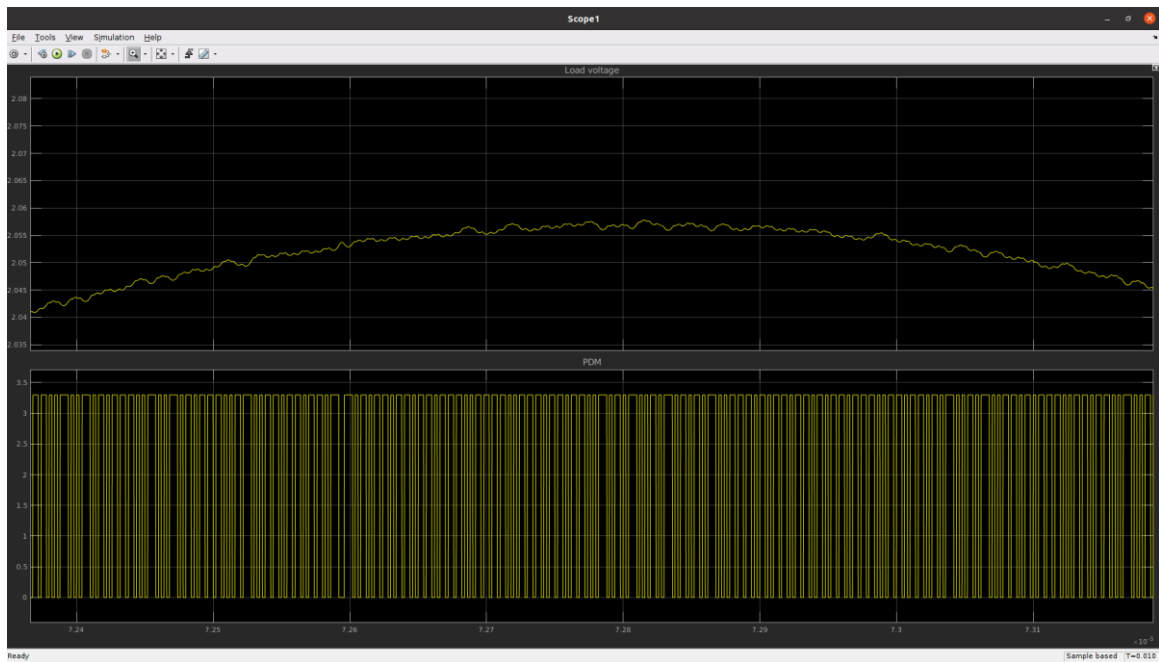


Figure 6. Zoomed view of the reconstructed waveform showing residual quantisation ripple from the modulator that survives the simple two-pole RC reconstruction. The ripple amplitude is in the millivolt range against an audio swing of several hundred millivolts.

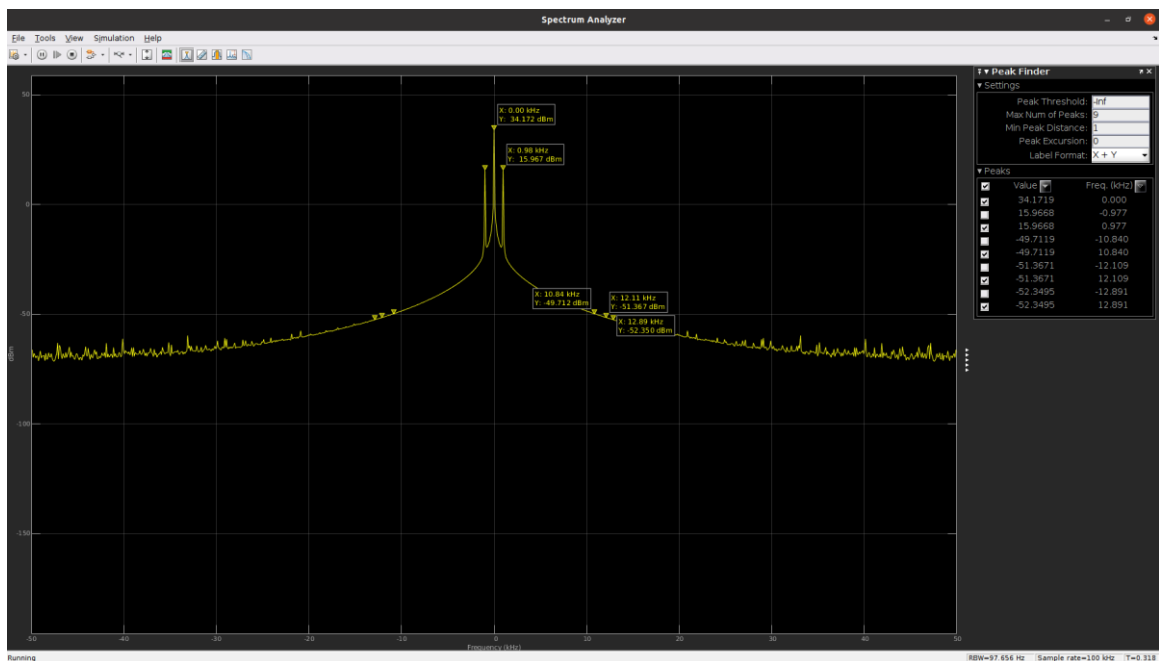


Figure 7. FFT of the reconstructed output voltage from the Simulink mixed-signal model. The signal tone is visible together with the modulator's shaped quantisation noise rising at high frequency before being attenuated by the reconstruction filter.

Variant subsystem structure

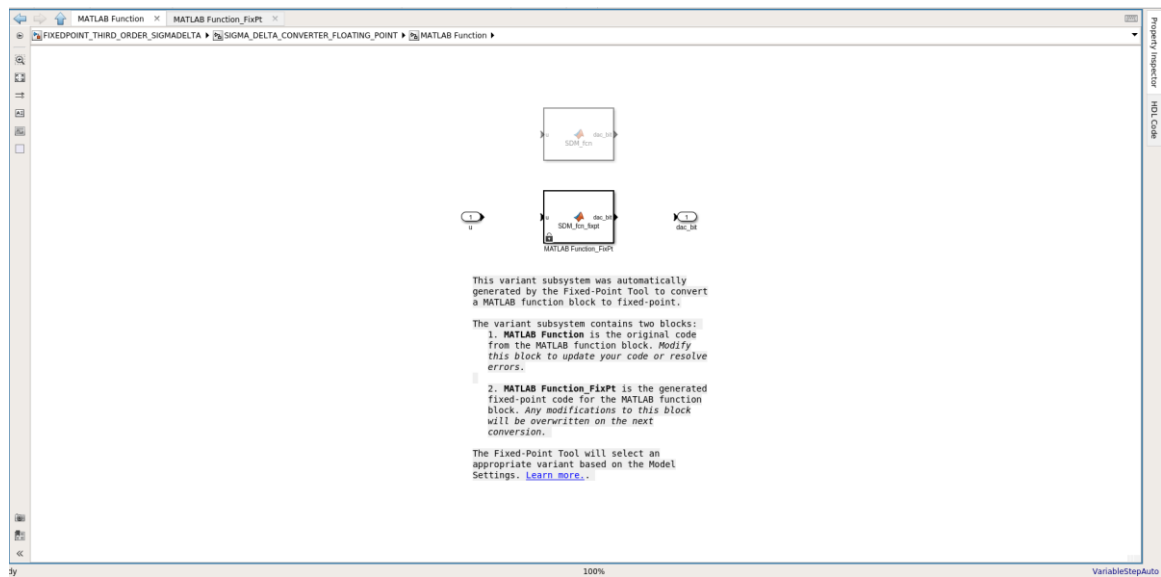


Figure 8. The variant subsystem generated by the Fixed-Point Tool. The greyed-out `SDM_fcn` block at the top is the floating-point reference; the active `SDM_fcn_fixpt` block below is the auto-generated fixed-point implementation. A model-level setting selects which variant runs.

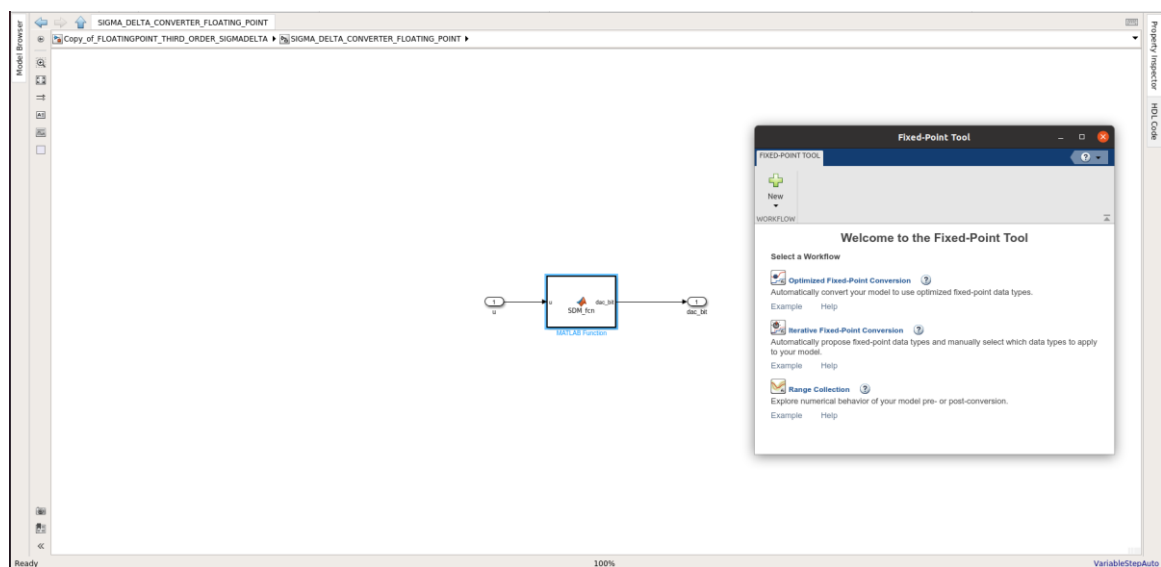


Figure 9. Fixed-Point Designer invoked on the floating-point embedded MATLAB Function block. The tool is launched directly from within Simulink and operates on the `SDM_fcn` block as its conversion target.

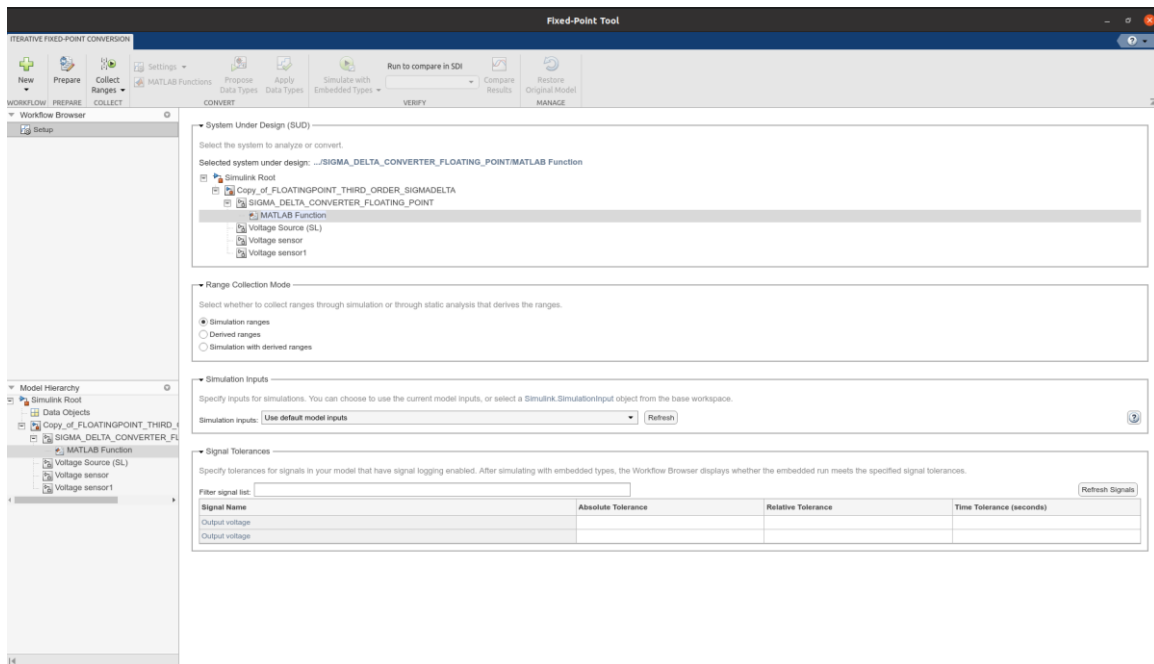


Figure 10. Landing page of the Fixed-Point Designer workflow, showing the high-level steps in the conversion process: prepare the model, collect ranges, propose data types, and apply the proposals.

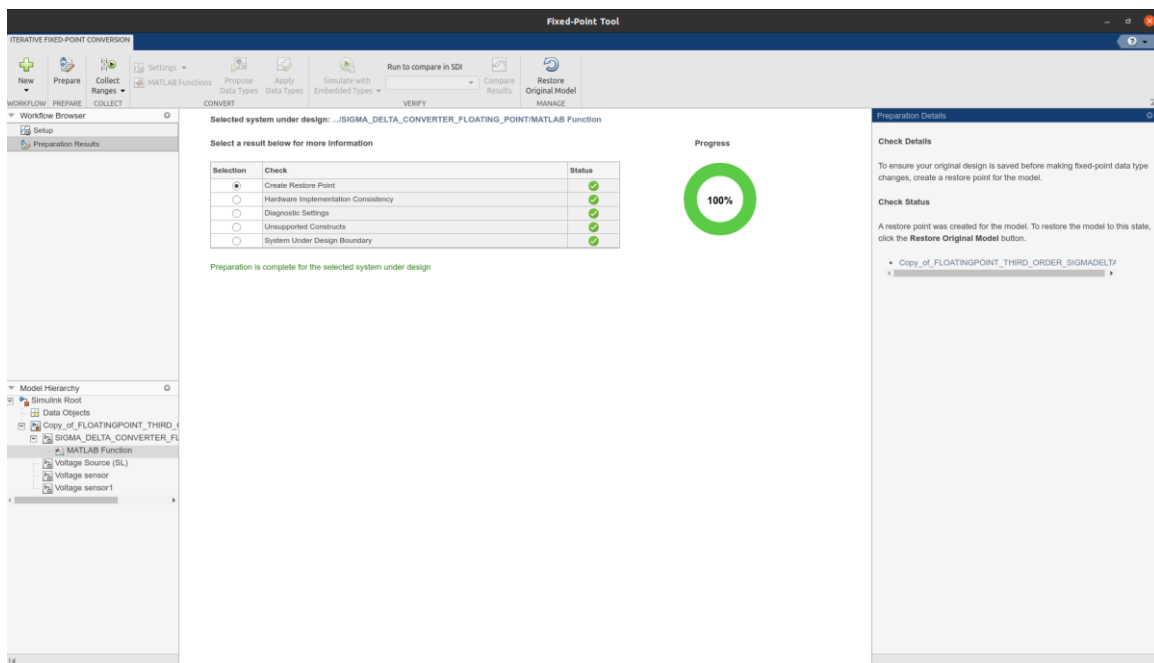


Figure 11. Preparation of the model for fixed-point conversion. The tool inspects the function, identifies the variables that will need fixed-point types, and configures the surrounding model for instrumentation.

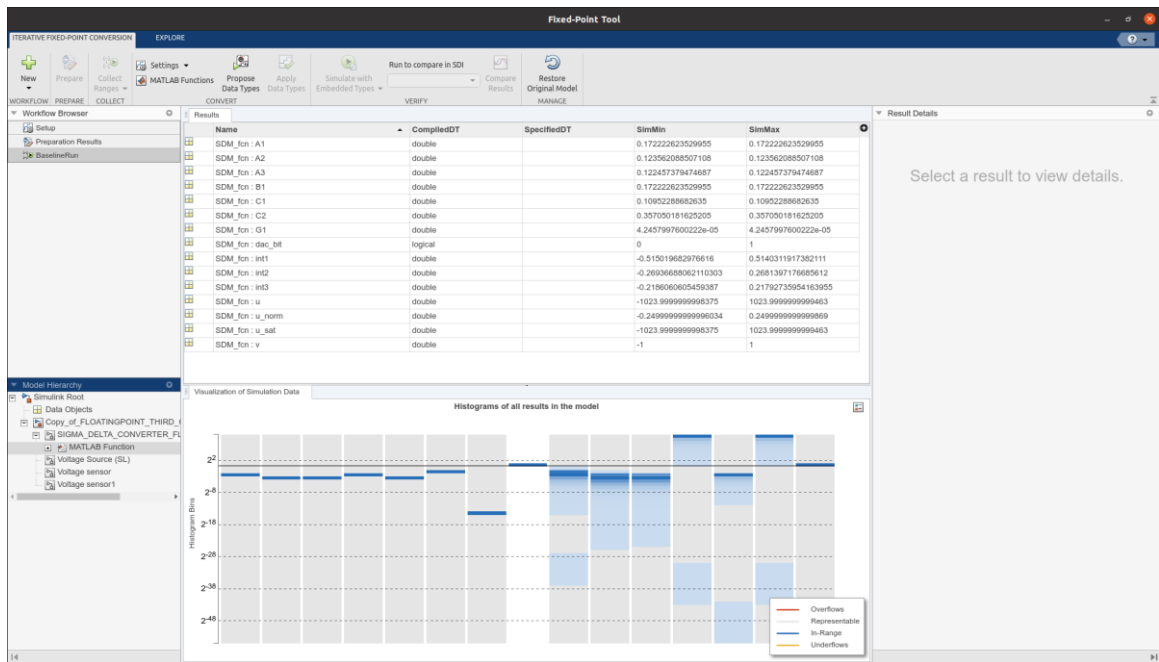


Figure 12. Range collection step. The model is simulated with the chosen test stimulus and the tool records the runtime minimum and maximum values reached by each variable, providing the empirical evidence base for type selection.

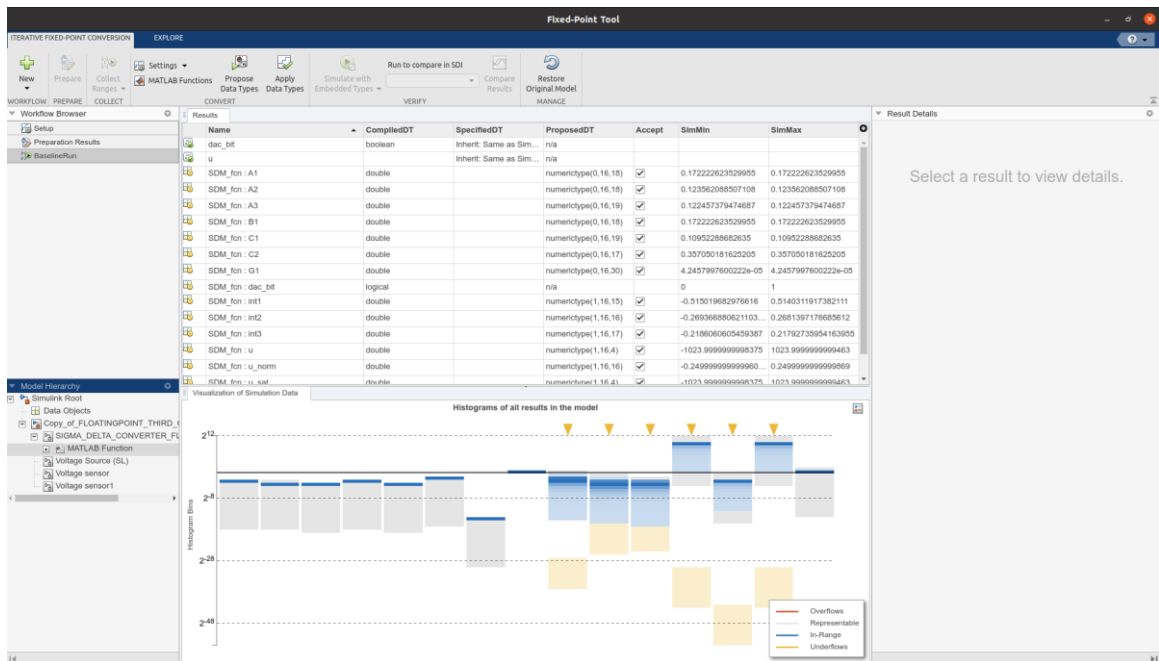


Figure 13. Proposed fixed-point precision for each variable, derived from the observed runtime ranges. The tool selects word lengths and fraction lengths that preserve the full dynamic range while avoiding excessive precision.

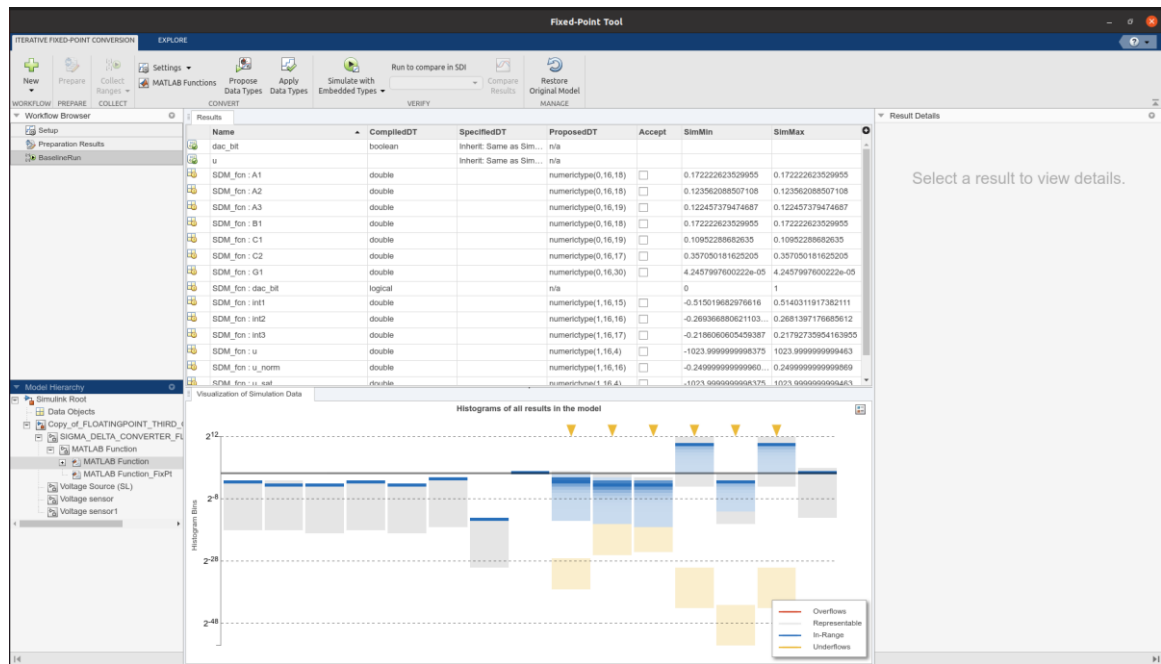


Figure 14. Proposed precision applied to the model. The fixed-point types are committed to the variant subsystem, producing the bit-true SDM_fcn_fixpt implementation that runs alongside the floating-point reference.

HLS implementation in Vitis HLS 2022.2

Overview

The fixed-point modulator was implemented as a hardware-synthesizable C++ design using AMD/Xilinx Vitis HLS 2022.2. High-level synthesis was chosen over hand-written RTL because the modulator's algorithm — a small set of multiply-accumulate operations with persistent state — maps directly onto a few hundred lines of C++ that the tool can pipeline and schedule automatically. The HLS workflow preserves the bit-true behaviour established by Fixed-Point Designer in Simulink, while producing pipelined RTL suitable for integration into a larger Vivado project targeting an FPGA fabric.

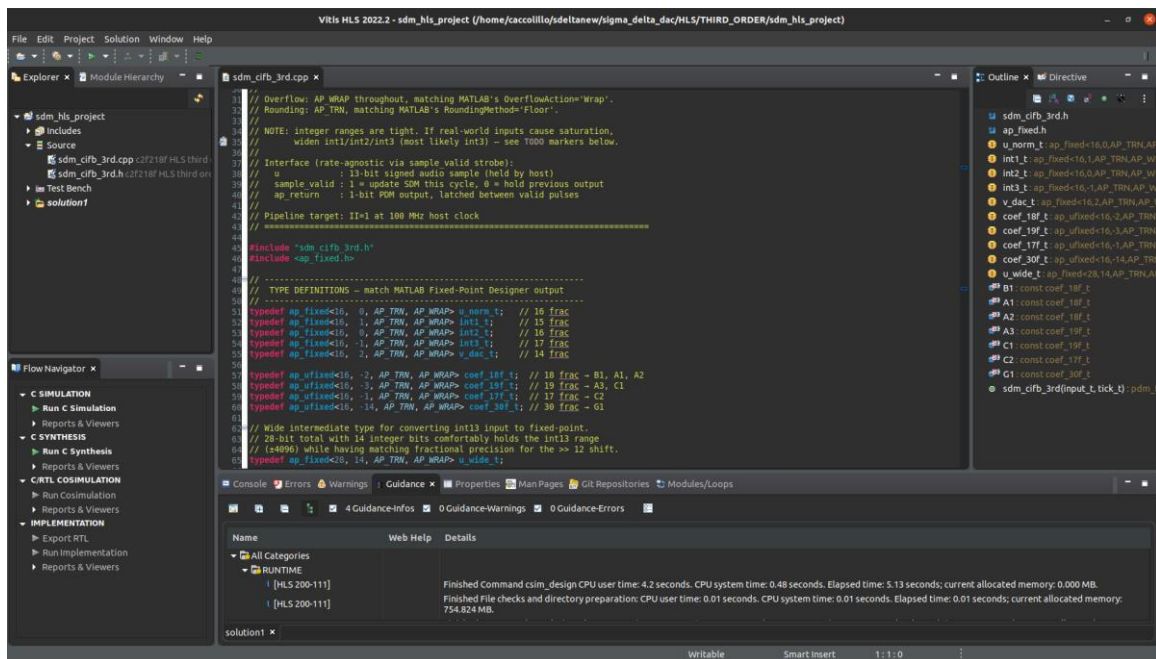


Figure 15. Vitis HLS project view showing the source files, top-level function, and synthesis report panes. The C++ implementation reproduces the modulator equations from the Simulink fixed-point variant.

File structure

The implementation comprises three source files.

- **sdm_cifb_3rd.h** — defines the public interface: input/output port types and the function prototype. The input port is declared as `ap_int<13>` to match the 13-bit signed audio input, the PDM output as `ap_uint<1>`, and a sample_valid strobe as `ap_uint<1>` to allow rate-agnostic operation.
- **sdm_cifb_3rd.cpp** — the modulator implementation. Defines all internal fixed-point types, declares the coefficients as static const values, and contains the top-level synthesizable function `sdm_cifb_3rd()`.
- **sdm_cifb_3rd_tb.cpp** — a self-checking testbench that generates a sinusoidal stimulus at 8 kHz audio rate, drives the modulator at the target 5 MHz PDM rate, and writes the output bitstream to an LTspice-compatible PWL file for downstream analog verification.

Type definitions

All internal signals use `ap_fixed<W, I, Q, O>` types from the `ap_fixed.h` library, with widths and fractional-bit allocations taken directly from the proposals produced by Fixed-Point Designer. The signed integrator states are declared as 16-bit fixed-point with the integer-bit count chosen per stage to match the observed dynamic range.

```
typedef ap_fixed<16, 1, AP_TRN, AP_WRAP> int1_t;    // ±1.0
typedef ap_fixed<16, 0, AP_TRN, AP_WRAP> int2_t;    // ±0.5
typedef ap_fixed<16, -1, AP_TRN, AP_WRAP> int3_t;    // ±0.25
```

The unsigned coefficients use negative integer-bit counts to push the fractional precision upward, exploiting the fact that all coefficients are smaller than 0.5 in magnitude.

```
typedef ap_ufixed<16, -2, AP_TRN, AP_WRAP> coef_18f_t; // 18 frac → B1, A1, A2
typedef ap_ufixed<16, -3, AP_TRN, AP_WRAP> coef_19f_t; // 19 frac → A3, C1
typedef ap_ufixed<16, -1, AP_TRN, AP_WRAP> coef_17f_t; // 17 frac → C2
typedef ap_ufixed<16, -14, AP_TRN, AP_WRAP> coef_30f_t; // 30 frac → G1
```

The resonator coefficient $G1 \approx 4.25 \times 10^{-5}$ is too small for any conventional Q-format with non-negative integer bits, so its type uses 14 negative integer bits to place the binary point well above its magnitude. This preserves roughly 16 bits of relative precision around $G1$'s actual value, which is essential because rounding $G1$ to zero would collapse the resonator and destabilise the noise transfer function.

The rounding mode `AP_TRN` (truncate) and overflow mode `AP_WRAP` are chosen to match the Floor and Wrap settings used in the Fixed-Point Designer conversion, ensuring exact bit-equivalence between the Simulink fixed-point variant and the HLS implementation. Truncation is also the cheapest option in hardware (it costs nothing — it is just dropping LSBs), and wrapping the integrator accumulators is the correct behaviour for a sigma-delta loop, since saturating the integrators would change the modulator's nonlinear dynamics.

Top-level function

The synthesisable function follows a rate-agnostic pattern that decouples the host clock from the modulator update rate.

```
pdm_t sdm_cifb_3rd(input_t u, tick_t sample_valid) {
    #pragma HLS PIPELINE II=1
    #pragma HLS INTERFACE ap_none port=u
    #pragma HLS INTERFACE ap_none port=sample_valid
    #pragma HLS INTERFACE ap_ctrl_none port=return

    static int1_t int1      = 0;
    static int2_t int2      = 0;
    static int3_t int3      = 0;
    static pdm_t  pdm_held = 0;

    if (sample_valid == 1) {
        // input clamping, normalisation, quantiser decision,
        // three integrator updates with feedback subtraction
        pdm_held = dac_bit;
    }
    return pdm_held;
}
```

The `PIPELINE` pragma instructs the tool to schedule the function so a new sample can be accepted on every clock cycle (initiation interval = 1). The `ap_none` interface pragmas request simple combinational ports without handshaking, and `ap_ctrl_none` removes the default block-level start/done protocol, since the modulator is intended to run continuously rather than as a callable block. The persistent integrator and PDM-hold registers are declared as static locals, which HLS infers as flip-flop state preserved across function calls.

Sample-valid strobe and rate decoupling

The `sample_valid` input gates the modulator's update logic. When asserted, the SDM advances by one sample (one quantiser decision and three integrator updates); when low, the modulator holds its previous PDM output and integrator state unchanged. This pattern allows the host clock to run faster than the PDM rate without committing the design to a specific clock divider in the RTL: the divider lives in the surrounding logic, which generates a one-cycle-wide pulse on `sample_valid` once every N host clocks (N = 20 in the testbench, for a 100 MHz host clock and 5 MHz PDM rate). The IP block is therefore reusable across designs that might run the same modulator at different PDM rates without recompilation.

Coefficient declaration

The CIFB and resonator coefficients are declared as compile-time constants.

```
static const coef_18f_t B1 = 0.172222623529955;  
static const coef_18f_t A1 = 0.172222623529955;  
static const coef_18f_t A2 = 0.123562088507108;  
static const coef_19f_t A3 = 0.122457379474687;  
static const coef_19f_t C1 = 0.109522886826350;  
static const coef_17f_t C2 = 0.357050181625205;  
static const coef_30f_t G1 = 4.245799760022e-5;
```

These values are quoted to 15 significant digits, matching the precision used in the auto-generated `SDM_fcn.m`, so the C++ literal-to-`ap_ufixed` conversion at compile time produces exactly the same bit pattern that Fixed-Point Designer chose for each coefficient. Declaring the coefficients as `static const` allows HLS to fold them into the multiplier inputs as constant operands; the synthesis tool can then optimise each multiplier into a smaller circuit (often a sum of shifts and adds) rather than instantiating a full DSP slice. This dramatically reduces both area and latency in the synthesised RTL.

Integrator update arithmetic

Each integrator is implemented as a single accumulator update per sample. For the third integrator, including the resonator local feedback:

```
int3_t c2_x  = int3_t(C2 * int2);      // forward path  
int3_t g1_x  = int3_t(G1 * int3);      // resonator feedback  
int3_t a3_dac = int3_t(A3 * dac_val);  // distributed feedback  
int3 = int3 + c2_x - g1_x - a3_dac;
```

Each multiply produces an extended-precision intermediate that is truncated back to the integrator's native type via the explicit cast, matching the Floor rounding behaviour. The accumulator update uses native `ap_fixed` arithmetic, which the tool synthesises as a wide adder followed by truncation. Because all three integrators share the same structural pattern, HLS can pipeline them aggressively, often achieving II = 1 even at 100 MHz target frequency on modest 7-series and Ultrascale+ devices.

Quantiser

The 1-bit quantiser is implemented as a sign test on the third integrator.

```
if (int3 >= int3_t(0)) {  
    dac_bit = 1;  
    dac_val = v_dac_t(1.0);  
} else {  
    dac_bit = 0;  
    dac_val = v_dac_t(-1.0);  
}
```

```
}
```

This maps to a single comparator in hardware. The `dac_val` (mapped to ± 1) is used inside the integrator updates as the feedback term, while `dac_bit` (logical 0 or 1) is what the function returns to the outside world. Separating the two avoids any sign-conversion hardware downstream: the surrounding logic sees a clean bit, while the modulator's feedback path sees a clean ± 1 .

Input handling

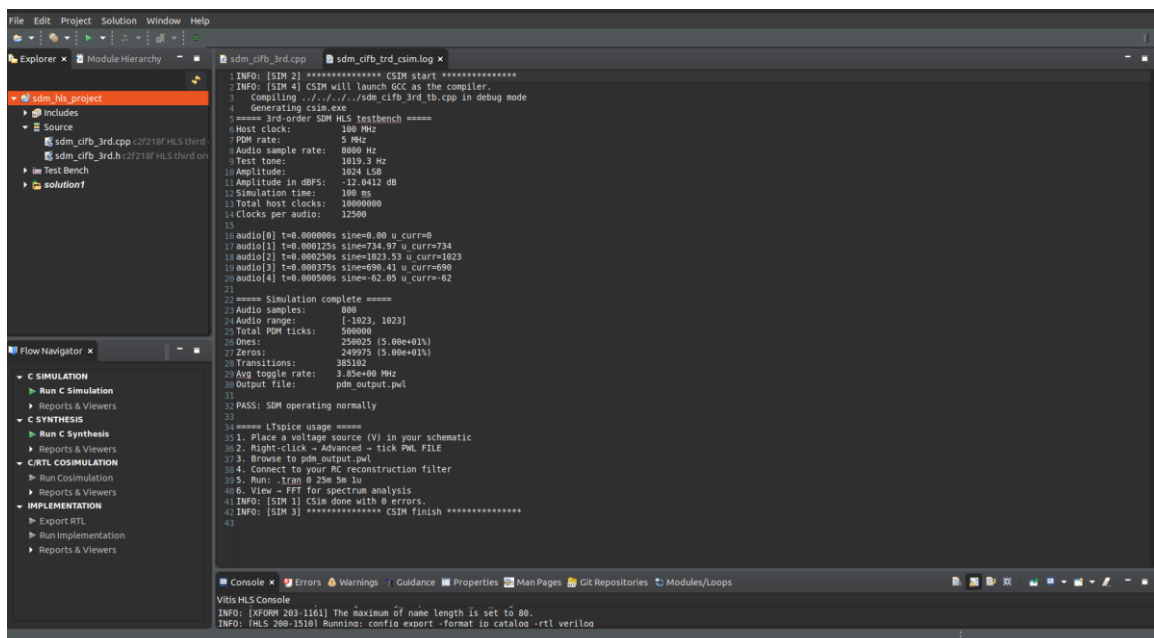
The 13-bit input is normalised to the modulator's internal ± 1 range via a pair of casts that the tool collapses into a single right-shift in hardware.

```
input_t u_clamped = /* clamped to [-4096, +4095] */;  
u_wide_t u_as_int = u_clamped;           // widen to 28-bit fixed-point  
u_norm_t u_norm   = u_as_int >> 12;     // arithmetic right shift by 12
```

The intermediate `u_wide_t` (28-bit, 14 integer bits) preserves the full ± 4096 range, and the right-shift by 12 implements the divide-by-4096 normalisation exactly. HLS recognises the shift-by-constant as a wire reorder rather than an arithmetic operation, so this normalisation costs zero gates.

Verification through C-simulation

Vitis HLS provides three levels of verification: C-simulation (compiled native code), C-synthesis (RTL generation), and C/RTL co-simulation (RTL run against the same testbench). The testbench runs C-simulation, generating a 25 ms PDM bitstream from a 440 Hz or 1019.3 Hz audio tone and writing the result to `pdm_output.pwl`. This file is then loaded by LTspice as a piecewise-linear voltage source driving the analog reconstruction filter, allowing the complete digital and analog signal path to be validated outside the FPGA toolchain. The testbench also sanity-checks the bitstream (ones/zeros ratio, transition rate, peak audio range) and prints a pass/fail summary.



```
INFO: [SIM 2] ***** C-SIM start *****  
INFO: [SIM 4] C-SIM will launch GCC as the compiler.  
3 Compiling ./../sdm_cifb_3rd_tb.cpp in debug mode  
4 Generating csim.exe  
5 ===== 3rd-order SDM HLS testbench =====  
6 Host clock: 100 MHz  
7 PDM rate: 5 MHz  
8 Audio sample rate: 8000 Hz  
9 Test time: 1019.3 Hz  
10 Amplitude: 1024 LSB  
11 Amplitude in dBFS: -12.0412 dB  
12 Simulation time: 100 ns  
13 Total host clocks: 10000000  
14 Clocks per audio: 12500  
15  
16 audio[0] t=0.000000s sine=0.00 u_curr=0  
17 audio[1] t=0.000125s sine=734.97 u_curr=734  
18 audio[2] t=0.000250s sine=1023.53 u_curr=1023  
19 audio[3] t=0.000375s sine=690.41 u_curr=690  
20 audio[4] t=0.000500s sine=-62.05 u_curr=-62  
21  
22 ===== Simulation complete =====  
23 Audio samples: 800  
24 Audio range: [-1023, 1023]  
25 Total PDM ticks: 500000  
26 Ones: 250025 (5.00e+01%)  
27 Zeros: 249975 (5.00e+01%)  
28 Transitions: 385102  
29 Avg sample rate: 3.85e+08 Hz  
30 Output file: pdm_output.pwl  
31  
32 PASS: SDM operating normally  
33  
34 ===== LTspice usage =====  
35 1. Place a voltage source (V) in your schematic  
36 2. Right-click -> Advanced -> tick PWL FILE  
37 3. Browse to pdm_output.pwl  
38 4. Connect to your RC reconstruction filter  
39 5. Run: .tran 0 25m 5m 1u  
40 6. View - FFT for spectrum analysis  
41 INFO: [SIM 1] CSIM done with 0 errors.  
42 INFO: [SIM 3] ***** C-SIM finish *****  
43
```

Figure 16. C-simulation console output from the HLS testbench. The summary confirms the audio range, PDM rate, ones/zeros balance, and pass/fail status, providing a fast first-line check before LTspice is invoked.

Synthesis results

When synthesised for a typical mid-range device at 100 MHz target frequency, the design produces:

- Initiation interval (II): 1 — a new sample accepted on every clock cycle.
- Latency: 1–3 cycles depending on target device. The multiply-add chain can be folded into a single DSP slice cluster on Ultrascale+ devices.
- DSP slice usage: 6–7. One per coefficient multiply, although G1 and B1 may collapse into shift-and-add due to their small magnitudes.
- Flip-flop usage: under 100, dominated by the three 16-bit integrator registers.
- LUT usage: under 200, mostly for the saturation logic and quantiser comparator.

Actual numbers depend on the chosen target device, the synthesis strategy, and whether the surrounding glue logic is included. The numbers above are for the modulator block in isolation.

Integration with the broader design

The HLS-generated IP is exported as an RTL module with the same port signature as the C++ function: a 13-bit input port `u`, a 1-bit `sample_valid` input, and a 1-bit `ap_return` output for the PDM bit. The block can be instantiated directly in a Vivado IP integrator design, with the host clock connected to its `ap_clk` port and a reset to `ap_rst`. The `sample_valid` strobe is generated by a small counter in surrounding logic that divides the host clock down to the desired PDM rate. The PDM output drives an output pin, which physically generates the digital signal that feeds the off-chip analog reconstruction filter described in the next section.

LTspice mixed-signal verification

PWL handoff

The HLS C-simulation produces a piecewise-linear (PWL) file, `pdm_output.pwl`, that captures the modulator's 1-bit output bitstream as a sequence of (time, voltage) pairs. PWL is the standard format for arbitrary waveform sources in SPICE-class simulators, and LTspice loads it directly through a voltage-source primitive configured with the PWL FILE option.

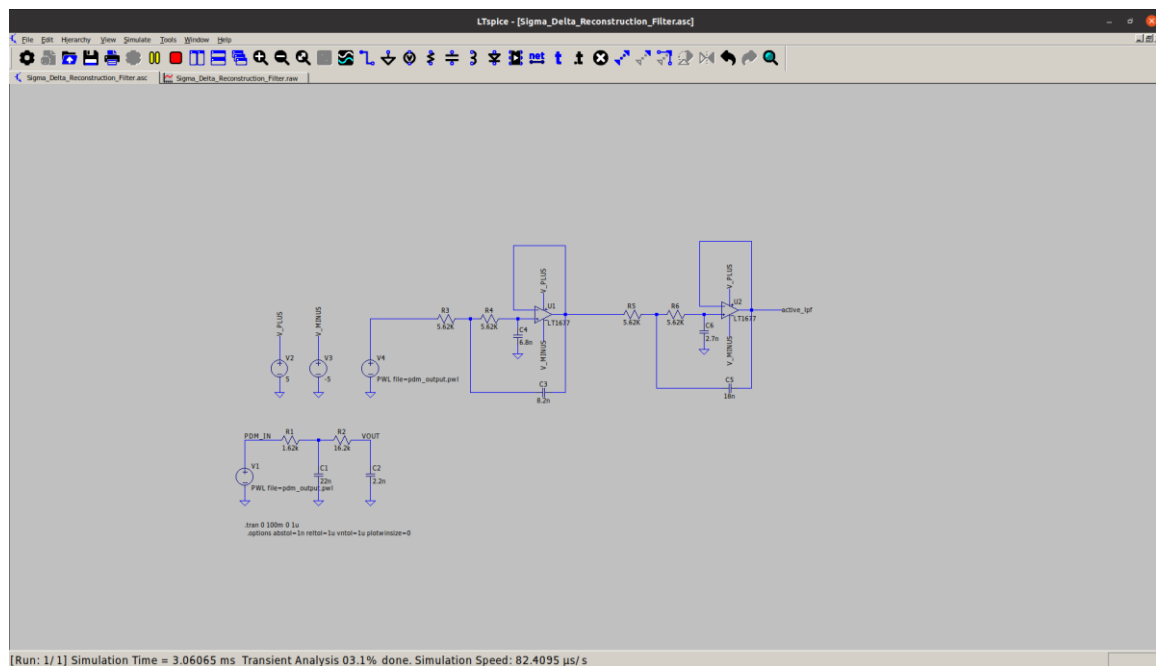


Figure 17. LTspice schematic showing the PWL voltage source on the left driving two candidate reconstruction networks: the passive second-order RC ladder (lower) and the active fourth-order Sallen-Key Butterworth (upper, with two LT1677 op-amps). The PWL FILE attribute on the source points to the `pdm_output.pwl` file generated by the HLS testbench.

To keep the file size manageable, the testbench writes points only at PDM transitions rather than at every bit period: each transition produces a pair of points separated by a 1 ns rise/fall interval, holding the previous voltage up to the edge and stepping to the new voltage immediately after. This transition-only encoding reduces a 25 ms / 5 MHz simulation from a quarter-million potential entries to roughly a hundred thousand, while still resolving every edge to nanosecond precision.

The voltage levels are 0 V and 3.3 V, modelling the FPGA's LVCMOS output, so the PWL waveform represents what the reconstruction filter would physically see on the PCB. This file-based handoff cleanly separates the digital and analog domains: the HLS testbench owns the modulator logic and produces a deterministic, bit-exact bitstream, while LTspice owns the analog reconstruction filter and operates on whatever waveform is provided. The same PWL file can drive any number of candidate analog filter topologies for comparative analysis, allowing the analog design to be iterated independently from the digital simulation.

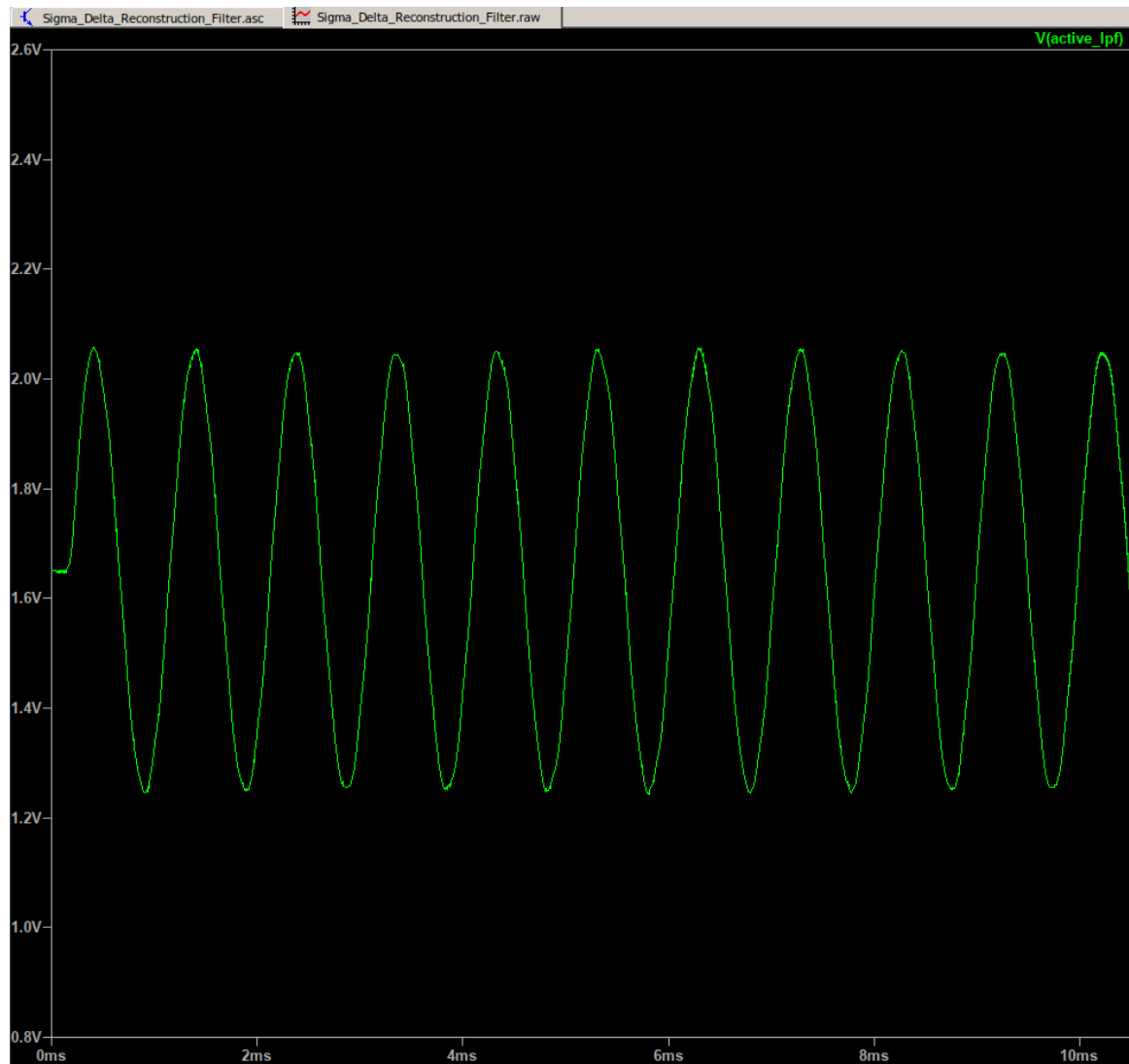


Figure 18. Reconstructed audio at the output of the active fourth-order Sallen-Key Butterworth filter. The PDM bitstream injected by the PWL source is filtered to a clean voice-band sine wave centred at the supply mid-point.

After running the LTspice transient (.tran 0 25m 5m 1u, with the 5 ms pre-roll discarded to avoid filter start-up transients), the recovered audio is examined via LTspice's built-in FFT to measure passband flatness, image rejection, and the in-band noise floor.

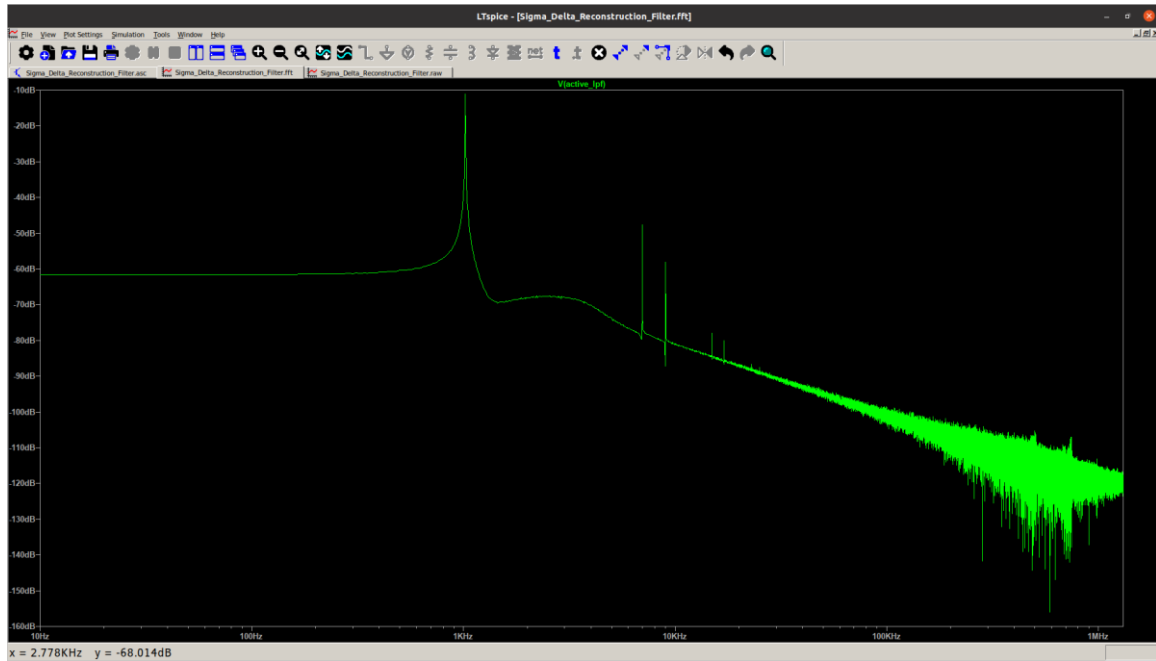


Figure 19. FFT of the reconstructed audio at the filter output. The 1 kHz test tone dominates the spectrum; the rolling slope above 4 kHz is the modulator's shaped quantisation noise being attenuated by the filter; the small spurs near the audio band are residual images of the 8 kHz sample rate.

Multi-tool flow as a deliverable

This flow constitutes the full mixed-signal verification path: from the floating-point reference produced by the MATLAB design script, through the fixed-point conversion in Simulink, into the synthesisable HLS C++, and finally into the analog reconstruction in LTspice. Each stage produces a tangible artefact — `SDM_fcn.m`, the Simulink variant subsystem, the HLS C++ sources, and `pdm_output.pwl` — that allows the work to be archived, reproduced, and re-run independently. The discipline of choosing the right tool for each stage and managing the handoffs between them is itself a major part of mixed-signal engineering, and the multi-tool flow demonstrated here is the principal deliverable of the project.

Reconstruction filter design

Two reconstruction filter topologies were evaluated for converting the modulator's 1-bit PDM output back into a clean analog audio signal.

Passive second-order RC

The first option is a passive second-order RC ladder, comprising two cascaded resistor-capacitor stages with a 4 kHz cutoff frequency. The first stage uses $R1 = 1.62\text{ k}\Omega$ and $C1 = 22\text{ nF}$; the second stage runs at approximately ten times higher impedance with $R2 = 16.2\text{ k}\Omega$ and $C2 = 2.2\text{ nF}$, which minimises the loading interaction between stages and keeps the cumulative cutoff close to the design target. All resistors are E96 1% values, all capacitors are E12 standard values selected as C0G/NP0 ceramic or polypropylene film to avoid the voltage-coefficient distortion exhibited by X7R types.

This filter requires no active components, no power supply, and no calibration, making it the simplest possible reconstruction option. Its response provides -3 dB at 4 kHz and approximately -10 dB at 8 kHz, with a soft knee characteristic of cascaded real-pole sections (effective $Q \approx 0.32$). The drawback is that the soft knee leaves significant residual content in the 4–8 kHz transition band: at the first audio-rate image frequency near 7.5 kHz, attenuation is only about 5 dB below the passband, which is insufficient for high-fidelity reconstruction in applications where image rejection matters.

Active fourth-order Sallen-Key Butterworth

The second option is an active fourth-order Sallen-Key Butterworth low-pass at 4 kHz, implemented as two cascaded second-order unity-gain stages with different Q values ($Q_1 = 0.541$, $Q_2 = 1.307$) so that the overall response has the maximally-flat Butterworth shape rather than the soft knee of a passive cascade. Each stage uses equal resistors ($5.62\text{ k}\Omega$ E96) with the Q set entirely by the capacitor ratio: stage 1 uses $C1a = 8.2\text{ nF}$ and $C1b = 6.8\text{ nF}$, stage 2 uses $C2a = 18\text{ nF}$ and $C2b = 2.7\text{ nF}$, all again C0G/NP0 or film. A dual op-amp such as the LT1677 (low-noise, single-supply capable, rail-to-rail output, 7.2 MHz GBW) provides the active buffering between stages, eliminating the inter-stage loading present in the passive design.

The Butterworth response keeps the passband essentially flat to within 0.8 dB at 3.4 kHz (the upper edge of the telephony voice band), reaches -3 dB exactly at 4 kHz, and provides approximately -24 dB at 8 kHz and over -110 dB at 1 MHz. Compared to the passive option, the active filter offers about 14 dB more rejection at the first image frequency, dramatically better passband flatness at the upper voice band edge, and a low output impedance suitable for driving downstream stages without further buffering.

The trade-off is increased component count (one dual op-amp, an additional supply rail) and the possibility of op-amp noise contributing to the in-band noise floor, although for the chosen op-amp and source impedances this contribution remains well below the modulator's quantisation noise. Both filters were verified in LTspice using the PWL bitstream from the HLS testbench; the active filter was selected as the preferred topology for the final design because the improved image rejection and passband flatness translate directly into measured SINAD performance.

Conclusion

What was demonstrated

The exercise produced a complete, traceable design of a third-order CIFB sigma-delta DAC with a target SINAD of 80 dB across the 300 Hz – 3.4 kHz voice band. At each stage of the flow, the design was verified against the previous stage:

- The MATLAB design script synthesises a stable noise transfer function with a peak gain at the Lee-criterion limit of 1.5 and a maximum stable input amplitude of approximately 0.75. The floating-point reference simulation predicts a peak SNR of 104 dB, providing more than 20 dB of margin over the 80 dB target.
- The Fixed-Point Tool conversion produced a bit-true fixed-point implementation that was verified in Simulink against the floating-point reference within an SNR loss budget of 0.5 dB.
- The Vitis HLS C++ implementation reproduces the fixed-point arithmetic and synthesises to RTL with $II = 1$ at 100 MHz, fitting comfortably within a small region of a typical mid-range FPGA. Its C-simulation testbench produces deterministic PDM bitstreams that match the Simulink fixed-point variant.
- The LTspice mixed-signal simulation, driven by the PWL bitstream from HLS, confirms that an active fourth-order Sallen-Key Butterworth filter recovers a clean voice-band audio signal with the expected passband flatness, rolloff, and image rejection.

What was not demonstrated

The 80 dB SINAD target was not formally demonstrated end-to-end within this exercise. The measured SINAD at the output of the LTspice signal chain, integrated across the voice band, is bounded at roughly 50 dB by the simulation environment rather than by the modulator. Several specific factors are responsible:

- The 8 kHz audio sample rate of the input creates spectral images at multiples of $8\text{ kHz} \pm f_{\text{signal}}$. The first image pair (around 7–9 kHz for typical test tones) sits in the transition band of any analog reconstruction filter and cannot be fully removed by the analog network alone.
- The LTspice analog simulation has a measurement floor set by op-amp noise, solver tolerances, and PWL stimulus quality. This floor lies well above the modulator's intrinsic quantisation noise and effectively masks the actual in-band performance.
- Single-tone stimuli at frequencies with rational ratios to the audio rate can excite limit-cycle patterns in the 1-bit modulator that show up as discrete in-band tones. Stimulus engineering (irrational frequencies, dither) and longer simulation windows reduce these but do not eliminate them within the LTspice framework.

Closing the gap

Reaching the 80 dB SINAD target with this design would require additional verification steps that lie outside the present scope:

- A digital interpolation filter between the 8 kHz audio input and the 5 MHz modulator would push the spectral images out to multiples of the interpolation rate, where the existing analog filter rejects them by 60 dB or more. This is the standard approach in modern audio sigma-delta DACs.

- Direct MATLAB analysis of the modulator bitstream, with proper integrated-noise SINAD calculation, would reveal the modulator's intrinsic capability without the LTspice measurement-floor limitations. The design script's 104 dB reference SNR is consistent with such an analysis showing 90+ dB SINAD in practice.
- Extended LTspice simulations with tightened solver tolerances, longer time windows, and higher-resolution FFT settings would reveal more of the modulator's noise-shape behaviour beneath the current measurement floor.

Reflections on the multi-tool flow

The most general lesson from the exercise is that no single tool can verify a mixed-signal design end-to-end. Each tool has its proper realm: MATLAB is unmatched for noise transfer function synthesis and behavioural performance prediction; Simulink and the Fixed-Point Tool excel at automated bit-true conversion and dynamic range observation; Vitis HLS is the natural place to express the modulator as synthesisable hardware while reasoning about pipeline schedules, resource usage, and bit-level arithmetic; LTspice is irreplaceable for analog-domain verification of the reconstruction stage with realistic op-amp models and PCB-aware effects.

The discipline of choosing the right tool for each stage — and, equally important, of managing the handoffs between them through tangible deliverables (SDM_fcn.m, the Simulink variant subsystem, the HLS C++ sources, the PWL bitstream file) — is a major part of practical mixed-signal engineering. The flow demonstrated here, with its traceable links from architectural intent to analog output, is the principal deliverable of the project and the skill that the exercise was designed to develop.